



جامعة خليفة
Khalifa University

Khalifa University of Science and Technology

COSC 788 – Advanced Computer Networks

Evaluation Report

Fall 2026

Student names and IDS:

Moza Alhmoudi – 100062840

Raaida Mahbub – 100063481

Mariam Almarzooqi – 100060686

Amna Albahrani – 100070037

Course Instructor: Dr. Hadi Otrok

Table of Contents

1. Introduction	3
2. Experimental Setup	3
2.1. Experimental Scenarios.....	3
2.2. Supporting Evidence	4
3. Experimental Results.....	4
3.1 Baseline Results	4
3.2 Effect of 2% Packet Loss.....	4
3.3 Effect of Network Delay	5
3.4 Effect of Window Size	5
4. Analysis and Discussion	6
5. Limitations and Possible Improvements	7
6. Conclusion.....	7

Team Member	Contribution
Moza Alhmoudi	Conducted and validated the experimental evaluation, including baseline, packet-loss and delay tests; performed the window-size comparison; reconciled sender/receiver results and verified file integrity; processed the final experimental results and prepared the evaluation figures and analysis.
Amna Albahrani	Developed and contributed to the core Java UDP file-transfer and reliability implementation, including protocol and transfer-control functionality.
Mariam Almarzooqi	Developed the metrics and logging functionality, including transfer measurements, RTT tracking and evaluation data collection.
Raaida Mahbub	Developed and integrated the LLM interaction functionality and network impairment/evaluation components, and contributed to integrated system testing and evaluation preparation.

1. Introduction

The aim of these experiments was to test the performance of the reliable UDP file transfer system under different network conditions. Since UDP does not provide reliability by itself, the implemented protocol uses ACKs, sequence numbers, timeouts and retransmissions to make sure that the file is transferred correctly.

The system was tested under three main conditions: a baseline with no added loss or delay, 2% packet loss, and 200 ms delay. Both a small and a large file were tested in each condition. An additional test was also performed with a larger sender window to see how the window size affects performance when the network delay is high.

The main measurements used in the evaluation were transfer time, throughput, retransmissions, packet drops, protocol overhead and RTT. File integrity was also checked after every experiment to make sure that the received file was identical to the original.

2. Experimental Setup

Before starting the experiments, the complete project was built and tested using Maven. All 868 tests passed with no failures or errors. Two files were then generated for the experiments. The small file was 262,267 bytes and the large file was 2,097,275 bytes. The same two files were used throughout the experiments so that the results from the different scenarios could be compared.

The integrated system also uses GPT-5-mini through the OpenAI Responses API for natural-language command interpretation and explanation of recorded transfer metrics. The LLM does not directly control network operations; the Java control plane remains responsible for deterministic command validation and transfer execution.

For the main experiments, the sender window was set to 8 packets (8,192 bytes) and the timeout was set to 500 ms. Each file was tested under the following conditions:

2.1. Experimental Scenarios

Table 1.

Scenario	Loss	Delay	Window	Timeout
Baseline	0%	0 ms	8 packets	500 ms
Packet loss	2%	0 ms	8 packets	500 ms
Delay	0%	200 ms	8 packets	500 ms
Window comparison	0%	200 ms	16 packets	500 ms

The baseline, packet-loss and delay experiments were performed using both file sizes. For the window-size comparison, the large file was transferred again under the 200 ms delay condition, but the window was increased from 8 to 16 packets. The timeout remained at 500 ms.

One important detail about the delay experiment is that the simulator applies the 200 ms delay separately to DATA packets and ACK delivery. Because of this, the measured RTT was expected to be around 400 ms rather than 200 ms.

After every transfer, the recorded metrics were checked and the sender and receiver results were reconciled. File integrity was also verified for every run.

2.2. Supporting Evidence

Raw experiment logs and reproduction commands are included with the project submission to support reproducibility of the reported results. Additional screenshots document the experimental runs and metric analysis. The accompanying demonstration recordings provide evidence of the natural-language transfer, controlled packet-loss scenario with collected metrics, LLM explanation grounded in recorded measurements, and deterministic rejection of an invalid command.

3. Experimental Results

Table 2 summarizes the results from the seven experiments. All transfers were completed successfully, and file integrity was verified in every run.

Table 2.

File	Scenario	Window	Time (s)	Throughput (Mbps)	Retransmissions	Drops	Mean RTT (ms)	Overhead
Small	Baseline	8	0.138	15.177	0	0	2.935	5.23%
Large	Baseline	8	0.445	37.713	0	0	1.195	5.04%
Small	2% loss	8	1.148	1.827	16	2	2.756	10.66%
Large	2% loss	8	18.137	0.925	280	42	1.588	16.20%
Small	200 ms Delay	8	13.512	0.155	0	0	407.640	5.23%
Large	200 ms Delay	8	104.518	0.161	0	0	406.054	5.04%
Large	200 ms Delay	16	52.553	0.319	0	0	406.340	5.04%

3.1 Baseline Results

Under the baseline condition, both files were transferred without any packet drops, retransmissions or timeout events. The small file took 0.138 s to transfer and reached a throughput of 15.177 Mbps. The large file took 0.445 s and reached a higher throughput of 37.713 Mbps.

The higher throughput for the large file is reasonable because the fixed parts of the transfer, such as starting and completing the transfer, have a larger effect on the total time of a very small file. In both cases, the protocol overhead remained close to 5%.

3.2 Effect of 2% Packet Loss

Adding 2% packet loss had a clear effect on the transfer performance. For the small file, throughput decreased from 15.177 Mbps to 1.827 Mbps, which is a decrease of about 88%. There were 16 retransmissions and 2 recorded packet drops. The overhead also increased from 5.23% to 10.66%.

The effect was larger for the large file. Its throughput decreased from 37.713 Mbps in the baseline to 0.925 Mbps with packet loss. The transfer recorded 42 packet drops and 280 retransmissions, while the overhead increased from 5.04% to 16.20%. Even with these retransmissions, the transfer completed successfully and the final file passed the integrity check.

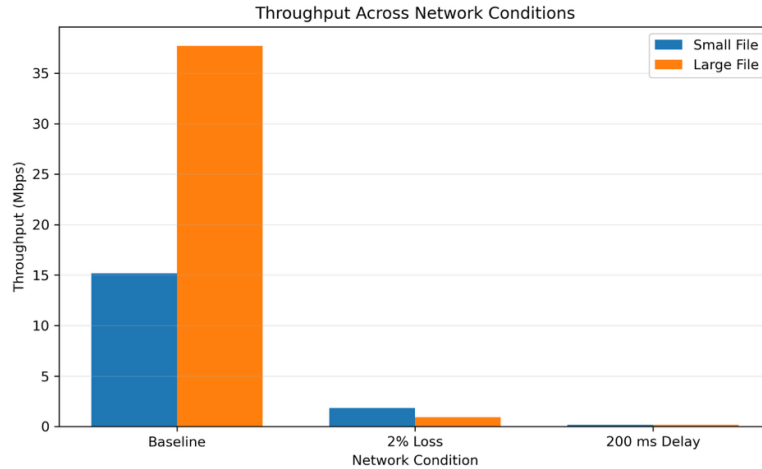


Figure 1: Throughput for the small and large files under the baseline, 2% loss and 200 ms delay conditions.

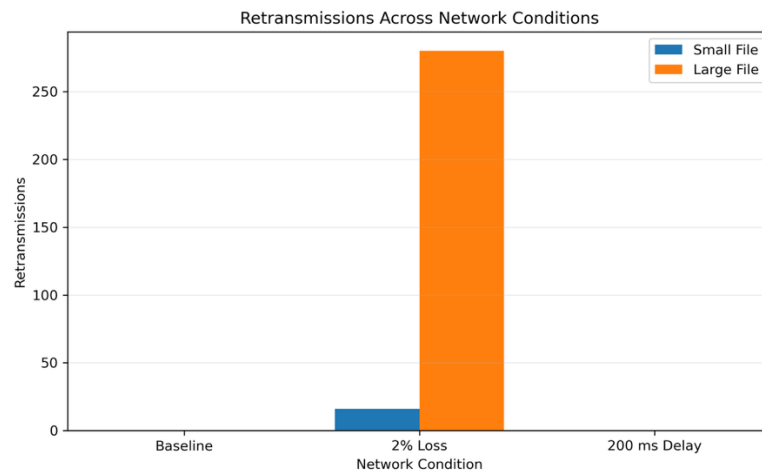


Figure 2: Number of retransmissions under the different network conditions.

3.3 Effect of Network Delay

The 200 ms delay had the largest effect on transfer time. For the small file, the transfer time increased from 0.138 s in the baseline to 13.512 s, while throughput decreased from 15.177 Mbps to 0.155 Mbps. The mean RTT increased from 2.935 ms to 407.640 ms.

A similar result was seen with the large file. Its transfer time increased from 0.445 s to 104.518 s, and throughput decreased from 37.713 Mbps to 0.161 Mbps. The measured mean RTT was 406.054 ms. This is close to the expected value because the 200 ms delay was applied separately to the DATA and ACK paths.

No timeout events or retransmissions occurred in either delayed transfer with the 500 ms timeout. Since the measured RTT was around 406–408 ms, the results also show why the timeout needs to account for the round-trip delay rather than only the one-way delay.

3.4 Effect of Window Size

To test the effect of the sender window under high delay, the large-file experiment was repeated with the window increased from 8 to 16 packets. The loss, delay and timeout settings were kept the same.

Increasing the window reduced the transfer time from 104.518 s to 52.553 s, which is a reduction of 49.72%. Throughput increased from 0.161 Mbps to 0.319 Mbps, an increase of 98.88%.

The mean RTT stayed almost the same, changing only from 406.054 ms to 406.340 ms. There were also no retransmissions or timeout events in either run. This shows that the improvement came from allowing more packets to be in flight while waiting for ACKs, rather than from any change in the network delay itself.

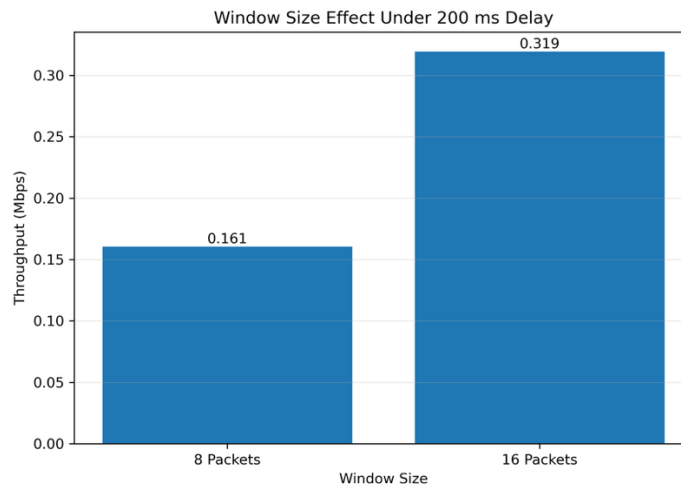


Figure 3: Throughput comparison using 8-packet and 16-packet windows under the 200 ms delay condition.

4. Analysis and Discussion

The experiments show that the protocol was able to maintain reliable file delivery under all of the tested conditions, but its performance changed significantly when loss or delay was introduced. All seven transfers completed successfully and passed the integrity check, including the runs with packet loss and high delay. This shows that the reliability mechanisms were able to recover from the impairments used in the experiments.

Packet loss mainly affected the system through retransmissions and additional protocol overhead. This was especially clear for the large file, where the 2% loss experiment resulted in 280 retransmissions and an overhead of 16.20%, compared with no retransmissions and 5.04% overhead in the baseline. The larger file was affected more because it required many more packets to complete the transfer, giving more opportunities for packet loss and recovery to affect the overall transfer.

Delay affected the system differently. The delayed experiments had no packet drops, retransmissions or timeout events, but their throughput was much lower than the baseline. The measured RTT was approximately 406–408 ms because the configured 200 ms delay was applied in both directions. With a window of 8 packets, the sender could only have a limited amount of unacknowledged data in flight before waiting for ACKs. As a result, the high RTT had a large effect on the overall transfer rate.

The window-size experiment shows how this effect can be reduced. Doubling the window from 8 to 16 packets almost doubled the measured throughput while the RTT remained practically unchanged. This allowed more data to be sent during each round of communication and reduced the amount of time the sender spent waiting for acknowledgements. However, this experiment only compared two window sizes, so it does not show that increasing the window will always continue to improve performance.

The timeout setting was also important in the delayed scenario. The mean RTT was slightly above 406 ms and the 95th-percentile RTT was approximately 409–414 ms. With the timeout set to 500 ms, no timeout events were recorded in the delayed runs. This suggests that 500 ms was suitable for the tested delay condition. A

timeout that is too close to or below the normal RTT could cause retransmissions even when packets have not actually been lost.

Overall, there is a trade-off between reliability and performance. The protocol successfully delivered the files even when packets were intentionally dropped, but recovering from these losses increased the number of transmitted packets, overhead and total transfer time. Under high delay, increasing the sender window improved throughput without changing the RTT or causing additional retransmissions.

5. Limitations and Possible Improvements

The experiments were performed in a controlled local environment using simulated packet loss and delay. This allowed the different configurations to be compared consistently, but it does not represent all conditions that may occur on a real network. The evaluation also covered a limited number of configurations: the main experiments used an 8-packet window and a 500 ms retransmission timeout, with one additional 16-packet window experiment under delay.

Another limitation is the use of Go-Back-N with cumulative acknowledgements. When recovery is triggered, multiple outstanding packets may be retransmitted even when some have already reached the receiver. This can increase retransmission traffic and protocol overhead under packet loss. This behavior is particularly relevant to the 2% loss experiments, where retransmissions and overhead increased substantially compared with the baseline.

The retransmission timeout is configurable but remains fixed during each experiment. The delayed experiments produced mean RTTs of approximately 406–408 ms, while the 500 ms timeout resulted in no timeout events. However, a fixed timeout may not perform equally well when RTT or jitter changes. A primary improvement would therefore be an adaptive retransmission timeout (RTO) based on measured RTT and RTT variation. This would allow the sender to adjust its timeout as network conditions change, potentially reducing premature retransmissions while still providing timely recovery from actual packet loss. Its effect could be evaluated using completion time, goodput, timeout events and retransmission ratio.

A second improvement would be to evaluate Selective Repeat as an alternative to Go-Back-N. Selective Repeat would allow correctly received out-of-order packets to be retained while retransmitting only missing packets, potentially reducing unnecessary retransmission traffic under lossy conditions. However, this would require additional receiver buffering and greater protocol complexity.

Future evaluation could also include additional window sizes, timeout values, variable delay and jitter, and combinations of loss and delay to determine how these mechanisms behave under a wider range of network conditions.

6. Conclusion

The experiments showed that the implemented UDP file transfer protocol was able to transfer both test files successfully under baseline, packet-loss and delayed network conditions. File integrity was verified in all seven runs, showing that the reliability mechanisms continued to work when network impairments were introduced.

Packet loss caused additional retransmissions and protocol overhead, while network delay had a large effect on transfer time and throughput. The window-size comparison also showed that increasing the sender window from 8 to 16 packets under the 200 ms delay condition increased throughput from 0.161 Mbps to 0.319 Mbps and reduced the transfer time by approximately 50%.

Overall, the results show that reliability can be added on top of UDP, but the performance of the protocol depends strongly on parameters such as the retransmission timeout and sender window size. The experiments also show the importance of selecting these parameters according to the network conditions rather than treating them as fixed values for every situation.